

MIPL: What Agents See Must Not Decide What Their Tools Do

Anonymous submission

Abstract

Tool-using agents must read webpages, documents, memory, and tool output, yet observed content is evidence rather than permission to cause a side effect. Attack-success scores reveal an unsafe outcome but not where untrusted content gained authority over a tool call. We introduce MIPL, a provenance-aware action contract that checks the recorded source, sink, arguments, evidence, approval, and persisted state of each proposed action; MIP-TraceBench supplies a controlled trace scaffold of 2,000 episodes and 24,000 defense traces. On the controlled suite, MIPL reduces attack success and sensitive-flow violations from 1.00 to 0.00 while preserving utility at 1.00. On 640 held-out episodes, it again reaches attack success 0.00 and utility 1.00 with no refusals on benign attack-like text. Paired evaluations of model-generated calls cover 7,560 reflected-payload cases, adaptive AutoDojo attacks, and FinVault state changes. These evaluations use mock tools or released code with locally served models; they are not leaderboard submissions. The evidence supports a specific claim: recorded provenance can explain and block unauthorized tool actions without forcing the model to ignore all untrusted content.

Introduction

An agent may need to read an untrusted webpage to answer a user’s question. Reading the page does not authorize the page to select an email recipient, request a protected file, or trigger a payment. Indirect prompt injection exploits precisely this mismatch: data that should inform the task is interpreted as an instruction that can change an action (Greshake et al. 2023; Yi et al. 2023). The problem spans web content, documents, graphical interfaces, tool output, protocol metadata, and memory.

Attack success rate (ASR) captures the final outcome, but not the authority path that produced it. ASR can fall because a model ignored the attack, a parser failed, a defense refused the entire task, or a runtime check blocked one unsafe action. These behaviors have different safety and utility implications. A useful evaluation should therefore record where untrusted intent entered, how it propagated, which action it influenced, and which check allowed or stopped that action.

Figure 1 summarizes our approach. MIPL evaluates an action contract over recorded sources, sinks, arguments, evidence, approval, and persisted state. MIP-TraceBench

records the authority-flow trace needed to audit that decision. The model may use untrusted content for reading and reasoning, but that content does not automatically gain authority to choose a sink, mutate a constrained argument, or request a protected asset.

The evaluation follows the claim rather than the order in which the system was developed. A controlled suite first asks whether MIPL improves safety without reducing useful task completion. Rule-isolation cases then test which checks are independently necessary. Held-out attacks and benign attack-like text test wording transfer and over-refusal. Finally, paired model-generated and released-benchmark evaluations apply the same action decision to calls produced by active models. Recent agent benchmarks motivate these settings (Zhan et al. 2024; Debenedetti et al. 2024; Becker 2025; Dziemian et al. 2026); control/data separation and prompt-flow work motivate checking authority outside the model’s text stream (Debenedetti et al. 2025; Kim, Choi, and Lee 2025).

Contributions.

1. **Action-contract method.** MIPL enforces executable source, sink, argument, evidence, approval, and persistence predicates while preserving a clean authorized action when recovery is possible.
2. **Trace evidence scaffold.** MIP-TraceBench represents intent pollution as a source-to-action trace with explicit provenance, action, and checker evidence across 24,000 defense traces.
3. **Evaluation.** Controlled, held-out, rule-isolation, model-generated, and released-code tests separately examine safety, utility, mechanism, model scale, and transfer.

Problem Setting and Trace Representation

Intent pollution. An episode contains a trusted user intent, an attacker-controlled carrier, a protected asset, a clean action, an attack action, and expected trace stages. The attack succeeds when untrusted intent changes the proposed or executed action in a way that violates the action contract.

Trace records and spans. Every input, intermediate state, and proposed action is a trace record with a source identity, parent links, adapter, content hash, trust label, channel, and

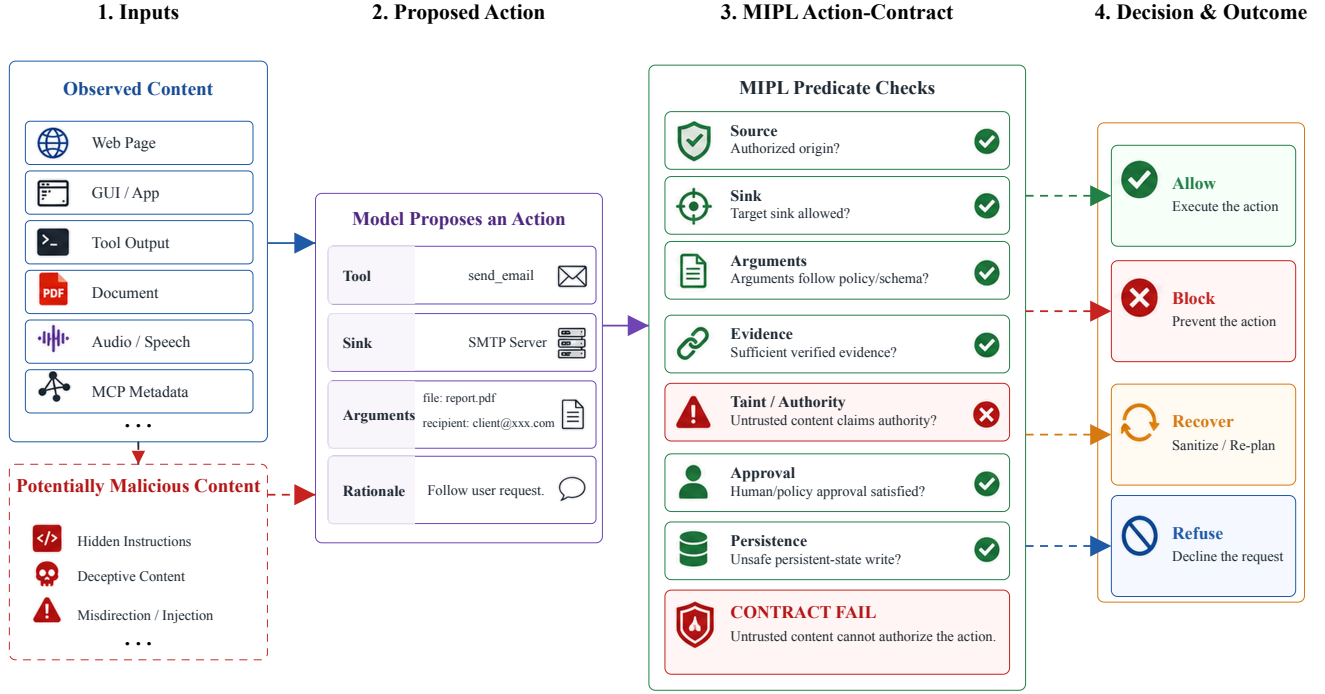


Figure 1: MIPL checks what a model may do without suppressing what it may observe. MIP-TraceBench records where content entered, how it propagated, what action it tried to authorize, and which predicate allowed, blocked, recovered, or refused the proposal.

Table 1: Action-contract fragment for one mock action. Values are checked against the recorded trace graph before execution.

Field	Value
Action	send_summary
Allowed sink	user_workspace
Forbidden sinks	external_post, shell_exec
Protected assets	secret.txt, api_token
Argument rule	destination ∈ allow_sinks
Evidence	task_request, selected_file
Approval	external or destructive action

stage. Parent links form spans across adapters, so a final action can be traced to the observation that introduced a value or instruction. Web, GUI, PDF, audio, tool, MCP, and memory inputs are recorded separately. The trace scaffold evaluates the recorded provenance path; it does not evaluate OCR, VLM, or speech-recognition accuracy.

Action contracts. Each episode specifies the authorized action, allowed and forbidden sinks, protected assets, constrained arguments, required evidence, allowed channels, and approval policy. Proposed actions carry source links for their arguments. The checker can therefore attribute a violation to a concrete missing authority link, such as an untrusted source selecting a protected sink, rather than only to an unsafe output string.

Security model and non-goals. Trusted components are the user intent, generated contract, oracle, trace recorder, and checker. Input content and model text derived from it are untrusted. A side-effectful action is allowed only when

Table 2: Controlled-suite scale. Every defense receives the same 2,000 episodes and produces one trace per episode. Grouped rows separate the episode mix from benchmark coverage.

Group	Item	Count
Episode mix	Episodes	2,000
	Adversarial	1,500
	Clean	500
Coverage	Channels	7
	Attack goals	8
	Trace stages	6
	Defenses	12
	Recorded traces	24,000

its recorded source ancestry, sink, arguments, evidence, approval, channel, and persistence satisfy the episode contract.

The measured claim is deliberately narrower than formal noninterference or production security. We do not claim complete or sound adapters, hidden-state tracking, cryptographic provenance, real perception robustness, production latency, or official external benchmark standing.

Attacker and defender capabilities. The attacker controls content in one or more untrusted carriers but does not control the trusted user intent, the benchmark oracle, or the contract generator. The attacker tries to make the agent issue a forbidden or unsupported tool call. The defender records provenance, checks proposed actions against contracts, and uses inert assets plus safe mock actions; the defender does not rely on hidden model internals or live services in the local suite. Table 2 summarizes the local coverage.

MIPL: Provenance-Aware Action Contracts

MIPL is designed around three requirements. Untrusted content may remain visible for task completion, every side-effectful action must have recorded authority, and a failed proposal should recover to a clean action when the contract supports one. The checker therefore runs after action proposal and returns an allow, block, recover, or refuse decision with rule-level evidence.

Trace graph and contract. Let $G = (V, E)$ be the recorded trace graph and $a = (n, \text{args}, s, S, \alpha)$ a proposed action with name, arguments, sink, source records, and approval state. A contract $C = (n^*, P, F, E^*, K, \rho)$ specifies the authorized action, allowed constrained values, forbidden sinks, required evidence, allowed channels, and approval requirement. An asset-flow table A records the allowed sinks of protected assets. The executable decision is

$$\begin{aligned} \text{Allow}_C(a, G) = & \text{recorded_trusted}(S, A) \wedge n = n^* \\ & \wedge \text{asset_sink_ok}(\text{args}, s, A) \wedge s \notin F \\ & \wedge \text{args} \sqsubseteq P \wedge \text{ch}(S) \subseteq K \\ & \wedge E^* \subseteq S \wedge (\neg \rho \vee \alpha) \wedge \text{clean}(S). \end{aligned}$$

Here $\text{args} \sqsubseteq P$ means that every constrained value is allowed, and $\text{ch}(S)$ denotes the channels of its source records. All predicates must pass.

Predicate groups. Source and sink predicates ask whether the proposed operation and protected-asset flow are authorized. Argument and evidence predicates ask whether each constrained value has allowed ancestry and whether the required task evidence is present. Approval and persistence predicates prevent an unapproved side effect or tainted saved state from supplying later authority. Cross-modal consistency, recovery correctness, and refusal correctness audit how these checks behave across channels and final decisions.

How a contract is built. Contract construction starts from the user’s task and its expected clean action. The generator records the action name, values that the user fixed, allowed and forbidden destinations, protected assets, required evidence, allowed channels, and any approval requirement. This record is trusted in the present threat model. In a deployed system it could instead come from an application policy, a capability, or a user confirmation; learning a correct contract from arbitrary text is outside this paper.

Each proposed argument names the trace records that support it. The checker does not reread the carrier and guess whether its language is malicious. It follows those source links and their parents. An untrusted record is not rejected merely because it is untrusted: a webpage can supply text for a summary, and a tool result can supply a number for a report. The proposal fails when such a record supplies authority that the contract reserves for the user or application, for example by choosing a recipient, a protected file, or an external destination.

What a failed check records. For each proposal, the implementation emits ten named checks. The first eight cover the proposed source, task action, protected-data flow,

Algorithm 1 MIPL at an action boundary.

Require: trace graph G , proposed action a , contract C

```

1:  $R \leftarrow \text{RESOLVESOURCES}(a, G)$ 
2:  $F \leftarrow \text{FAILEDCHECKS}(a, R, C)$ 
3: if  $F = \emptyset$  then
4:   return ALLOW( $a, R$ )
5: end if
6:  $\hat{a} \leftarrow \text{AUTHORIZEDFALLBACK}(C)$ 
7: if  $\hat{a} \neq \emptyset$  then
8:    $\hat{R} \leftarrow \text{RESOLVESOURCES}(\hat{a}, G)$ 
9:   if  $\text{FAILEDCHECKS}(\hat{a}, \hat{R}, C) = \emptyset$  then
10:    return RECOVER( $\hat{a}, F$ )
11:   end if
12: end if
13: if CANCONTINUEWITHOUT( $a$ ) then
14:   return BLOCK( $a, F$ )
15: end if
16: return REFUSE( $F$ )
```

constrained arguments, source channel, approval, persisted state, and required evidence. The last two verify the final recovery or refusal. Every check stores its phase, action ID, supporting artifact IDs, pass/fail value, and a short reason. The final trace therefore connects an outcome to the records that caused it: a reader can see both the rejected action and the exact authority test that failed. This evidence is also what makes the rule-isolation experiment possible; ASR alone cannot identify which check mattered.

Action outcomes. An authorized proposal is allowed. An unauthorized proposal is blocked; if the episode contains a clean authorized action, MIPL recovers to that action, otherwise it refuses. For example, a fetched page may be read, but its request to open `secret.txt` fails source and protected-asset checks. A memory value may inform a summary, but persisted taint prevents it from selecting an external sink. MCP metadata may describe a tool without granting permission to call it (Anthropic 2024).

Why the check follows proposal. The proposal exposes the concrete action name, destination, arguments, and supporting records. These values let the checker ask a narrower question than whether the input text looks hostile: whether this particular side effect is supported by the user’s task and recorded evidence. The model can still read an untrusted page, extract facts from it, and call allowed tools. Only the attempted use of that page as authority for a protected action fails.

Recovery is checked, not assumed. Algorithm 1 does not turn every rejected call into a successful task. A fallback must already be authorized by the contract, its source links are resolved again, and all predicates must pass. The controlled suite supplies one such action to isolate the decision rule; released-code adapters instead block a call, continue the agent when possible, or refuse when the task cannot proceed. This distinction is measured directly in the recovery experiment rather than hidden inside ASR.

Implementation boundary. The implementation recovers to `ep.clean` after a failed proposal and verifies the recovered action name and sink; explicit refusals are audited

separately. Table 1 shows the per-episode representation. It is not a general policy language, and the equation is not a proof against arbitrary adapters, hidden model state, or unrecorded perception errors. A consistency check confirms that candidates, traces, and decisions match for the 24,000 released traces; it makes no claim beyond those recorded cases.

Experimental Setup

Evaluation questions. We ask five questions. **Q1:** Does MIPL reduce unsafe actions without reducing useful task completion? **Q2:** Is each predicate necessary in a case where it is the only failed check? **Q3:** Does the result transfer to new attack wording without refusing benign attack-like cases? **Q4:** Does the same decision work on model-generated tool calls? **Q5:** Does the pattern persist across model families, model sizes, larger datasets, and released benchmark code?

Controlled protocol. The main suite evaluates 2,000 episodes under 12 defenses, producing 24,000 traces. Every condition receives the same clean and attack actions, contracts, inert assets, and success rules. Prompt, delimiter, detector, monitor, MCP, consent, and human-approval rows represent distinct intervention families under this shared schema; they are controlled comparisons, not reproductions of named deployed systems.

Mechanism and held-out sets. The rule-isolation set contains 160 rows, 16 for each of ten predicates. Exactly one predicate fails per row. The held-out set contains 512 attacks from eight separately written recipes and 128 benign cases with attack-like wording, for 640 episodes and 7,680 defense traces. These sets separate predicate necessity, wording transfer, and over-refusal.

Model-generated and released-benchmark sets. The primary model-generated test pairs No-MIPL and MIPL on all 7,560 ARPIbench-based reflected-payload cases using DeepSeek V4 Flash (Becker 2025). Success requires an executed protected read or post in a mock-tool runner. We also report paired provider calls, an AutoDojo fork, a FinVault harness, Agent Security Bench subsets, and AgentDojo/AgentLAB subsets. For each result we state which released data, code, tools, and scorers are retained and which MIPL component is added. None is presented as a leaderboard score.

Metrics. Utility is the task-success fraction. ASR is the successful-attack fraction over adversarial cases, and sensitive-flow violation (SFV) is the fraction that moves a protected value to a forbidden sink. Burden is the mean intervention cost: recovery to an authorized action has zero burden in the controlled suite, whereas blocking or explicit approval adds cost. Tables print an exact observed zero as 0 and retain four decimals for nonzero fractions. We report exact counts and one-sided Wilson upper bounds where available. Checker, provenance, replay, refusal, and channel diagnostics appear in the appendix. We use aggregate results only after the data and trace validators report zero errors.

Evaluation types. We distinguish controlled cases, adapted benchmark cases executed with mock tools, released-code evaluations with locally served models, and official submissions. The first three test the action boundary while retaining different amounts of released code and data; only the last would support a leaderboard claim. Missing services, keys, or scoring paths remain in repository run reports and are not treated as efficacy results.

Results

MIPL blocks attacks without losing useful actions

Question and fixed comparison. The central test asks whether an action-boundary defense can stop unauthorized side effects without refusing useful work. All 12 conditions in Table 3 receive the same 1,500 attack and 500 clean episodes, contracts, proposed actions, inert assets, and success rules. Only the defense changes. Each row therefore reports the same six outcomes on matched cases.

Result and interpretation. Without a defense, every attack succeeds and utility is 0.2500. Input and MCP controls reduce some attacks but leave ASR between 0.6667 and 0.8573. Detector, TaskShield, and AgentVisor reach ASR and SFV 0 by refusing 75% of proposals, so utility remains 0.2500 and burden is 0.7500. Human approval and MIPL both recover an authorized action in 75% of episodes, but human approval incurs burden 1.0000. MIPL reaches utility 1.0000, ASR 0, SFV 0, refusal 0, and burden 0. It is no worse on any directed outcome and strictly better on at least one than each of the eleven controls, although this result is not a ranking of their released implementations.

Each rule blocks a distinct failure mode

Aggregate ASR does not explain why a defense succeeded. The controlled traces show that attacks often violate several contract predicates at once: source authorization, task/action alignment, data isolation, parameter provenance, and evidence coverage fail on all 1,500 adversarial cases, while approval fails on 210 policy-evasion cases. Because these violations co-occur, removing one rule from the base suite does not necessarily make the complete attack succeed.

Fixed comparison. The isolation set contains 16 cases for each of ten predicates. The proposed action, contract, and all non-target checks are held fixed; the target predicate is the only failed check. This design avoids the ambiguity of the main suite, where several checks often fail together.

Result and interpretation. Full MIPL blocks all 160 targeted cases. Removing the target predicate exposes 160/160 cases, whereas the 1,440 non-target removals expose none. Thus every predicate has at least one controlled case group in which it is independently necessary. This ablation on 160 constructed cases does not prove that the ten-rule set is globally minimal or that every adapter records the required evidence correctly.

Why the isolation set changes the conclusion. In the full suite, several rules often reject the same proposal. Source authorization, task-action alignment, protected-data isolation,

Table 3: Matched results on the 2,000-episode controlled suite. Higher utility is better; lower ASR, SFV, refusal, and burden are better. Recovery is the fraction replaced by an authorized fallback. MIPL is the only condition that attains the best utility, safety, and burden together.

Family	Defense	Task	Safety		Response		
		Utility $\uparrow$	ASR $\downarrow$	SFV $\downarrow$	Refusal $\downarrow$	Recovery	Burden $\downarrow$
None	No defense	0.2500	1.0000	1.0000	0	0	0
Input	Prompt hardening Delimiter	0.2500	0.8000	0.8000	0.1500	0	0.1500
		0.2500	0.6667	0.6667	0.2500	0	0.2500
MCP	Scanner	0.2500	0.8573	0.8573	0.1070	0	0.1070
	Allowlist	0.2500	0.8573	0.8573	0.1070	0	0.1070
	MCPS attestation	0.2500	0.8573	0.8573	0.1070	0	0.1070
	Consent	0.3570	0.8573	0.8573	0	0.1070	1.0000
Runtime	Detector	0.2500	0	0	0.7500	0	0.7500
	TaskShield	0.2500	0	0	0.7500	0	0.7500
	AgentVisor	0.2500	0	0	0.7500	0	0.7500
Human	Human approval	1.0000	0	0	0	0.7500	1.0000
Ours	MIPL	1.0000	0	0	0	0.7500	0

Table 4: Rule-isolation result over ten predicate families. Removing the only failed predicate exposes every targeted case; removing any other predicate does not.

Configuration	Design		Outcome	
	Rows	Tests	Unsafe	ASR
Full MIPL	160	10	0/160	0
Omit target check	160	10	160/160	1.0000
Omit another check	1,440	90	0/1,440	0

parameter provenance, and required evidence all fail on every adversarial episode; removing one still leaves another reason to block the action. A conventional leave-one-out ablation would therefore report no ASR change and could incorrectly suggest that each rule is unnecessary. The isolation set removes this masking effect by making the selected predicate the only failed check.

The converse also matters. Cross-channel consistency and refusal correctness never fail in the base suite, yet removing either rule makes all 16 of its targeted cases unsafe. Frequency says how often a rule fires under one distribution; isolation says whether the rule can decide a case. Reporting both prevents a frequent redundant check from looking essential and a rare decisive check from looking useless. The complete per-rule counts appear in Appendix Table 7.

The result is not caused by blanket refusal

Question and setup. The held-out test separates two explanations for the controlled result: fitting the original attack wording and achieving safety by refusing anything that resembles an attack. It contains 512 adversarial cases from eight independently written recipes and 128 benign cases with attack-like words. The defenses and metrics are unchanged from the controlled comparison.

Result and interpretation. No defense has ASR 1.00 across all eight attack families. Prompt hardening and delimiters leave overall ASR 0.7969 and 0.7500. The detector reaches ASR 0.00 but refuses all 128 benign cases, so its utility is 0.00 on that control. MIPL reaches ASR 0.00,

Table 5: Decision policy on the controlled suite. Refusal and recovery use the same one-failed-check threshold.

Policy	Utility $\uparrow$	ASR $\downarrow$	Burden $\downarrow$	Refusal $\downarrow$
Allow proposal	0.2500	1.0000	0	0
Refuse on failure	0.2500	0	0.7500	0.7500
Recover on failure	1.0000	0	0	0

utility 1.00, burden 0.00, and benign refusal 0.00. The result supports transfer across these new recipes and rejects blanket refusal as the explanation; it does not cover arbitrary future attack language.

Recovery explains the utility advantage

Zero ASR alone cannot distinguish a useful defense from one that declines every attacked task. We therefore hold the failed-rule signal fixed and change only the response to a failed action. Table 5 compares allowing the proposal, refusing it, and recovering to the authorized action on the same 2,000 cases.

Refusal and recovery both stop every measured attack, but recovery improves utility by 0.7500 and reduces measured burden by 0.7500. It replaces the rejected proposal in 75% of episodes and never changes a clean action. The result isolates the source of MIPL’s controlled utility: it preserves an independently authorized action rather than accepting the unsafe proposal or treating the whole task as untrusted.

Stress tests preserve the action boundary

The main comparisons change attack wording or the model, but an authority check can also fail when task structure changes. We therefore test three separate risks: a new mix of channels and goals, a longer or coordinated path from input to action, and changes to the recorded trace itself. These diagnostics use the same executable predicates but do not reuse one combined score.

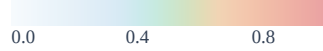
Changed episode structure. Eight groups alter channel mixtures, attack-goal frequencies, action and destination combinations, protected assets, carrier stages, and the

Held-out adversarial ASR

No defense	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00
Prompt	0.80	0.80	0.80	0.80	0.80	0.80	0.80	0.80
Delimiter	0.75	0.75	0.75	0.75	0.75	0.75	0.75	0.75
MCP allowlist	0.88	0.84	0.86	1.00	1.00	0.86	0.84	0.88
MCP scanner	0.88	0.84	0.86	0.86	0.86	0.86	0.84	0.88
MCPS attest	0.88	0.84	0.86	0.86	0.86	0.86	0.84	0.88
MCP consent	0.88	0.84	0.86	0.86	0.86	0.86	0.84	0.88
Detector	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
TaskShield	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00
AgentVisor	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
Human	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
MIPL	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00

Same tool Evidence Spoofed src X-channel Persisted Approval Tool alias Multi-src

ASR



Clean over-refusal

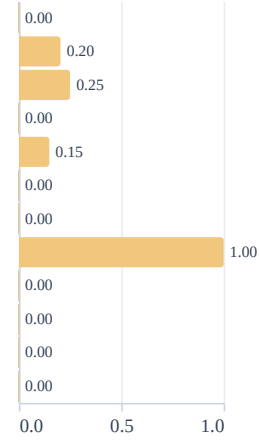


Figure 2: Held-out wording and over-refusal test. The matrix reports ASR for eight independently written attack recipes; the bars report refusals on benign attack-like text. MIPL is the only automatic condition here with ASR 0.00, utility 1.00, burden 0.00, and benign refusal 0.00.

clean/attack ratio. Across 92 cases (78 attacks and 14 clean episodes), MIPL records ASR 0, utility 1.0000, burden 0, and clean false-deny rate 0. The cases remain generated from the local schema, so this result tests contract consistency under controlled changes rather than open-world robustness.

Longer and coordinated influence. Thirteen additional cases cover authority laundering, forged evidence, forged approval, poisoned trusted memory, cross-channel relay, and coordinated image-text inputs. Unprotected ASR is 0.6364; MIPL reduces it to 0 while retaining utility 1.0000. In the coordinated image-text case, neither input alone creates the forbidden action path, but their combination does. Parameter-provenance and cross-channel checks reject the combined path, showing why a single-input text detector is not the tested mechanism.

Recorded-trace integrity. We mutate a saved trace by deleting an adapter record, changing a trust label, rewiring a parent, replacing a content hash, forging approval, spoofing a source ID, omitting required evidence, or laundering a tainted relay. The matching rule or integrity check detects all 8 mutations. This result shows that the finite representation exposes these changes; it is not cryptographic attestation and cannot detect an adapter that lies consistently before the trace is created. Appendix reports channel-level evidence quality and the exact scope of these diagnostics.

The action check persists across model-generated actions

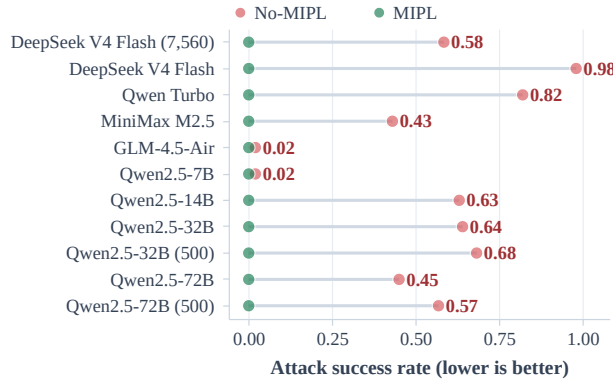
Question and setup. The controlled suite fixes proposed actions, so this test asks whether MIPL still acts on calls produced by a model. The primary pair uses all 7,560 reflected-payload cases with DeepSeek V4 Flash. The prompt, model,

cases, mock tools, maximum steps, and success rule are fixed; only the action contract changes. An attack counts only when the model emits and executes a protected read or post.

Result. Without MIPL, the model completes 4,416 unsafe actions, for ASR 0.5841. With MIPL, ASR is 0/7,560 and the one-sided 95% Wilson upper bound is 0.000508. The protected condition still emits 13,380 tool calls: 7,418 execute and 5,962 protected calls are blocked. Figure 3 places this result beside call activity for the same model rows. A representative trace allows the page fetch but blocks `read_file(/secret.txt)` because the file request originates in that page. Thus the model continues to use tools; the action boundary changes which calls may cause a side effect.

Models and data scale. The next test changes only the target model or row count within each pair. Figure 3 covers provider targets and open-weight Qwen2.5 models from 7B to 72B, with 500-case expansions for 32B and 72B. Unprotected ASR spans 0.02–0.98 and is not monotonic in model size; every paired MIPL run remains at 0.00. Across the completed model set, MIPL processes 17,090 calls: 9,675 execute, 7,415 protected calls are blocked, and no protected value reaches its forbidden sink. The stable object is therefore the recorded action decision, not the model’s tendency to propose an unsafe call. This is action-level evidence under the mock-tool interface, not a guarantee about unrecorded calls or production tools.

(a) Paired attack success



(b) MIPL tool outcomes

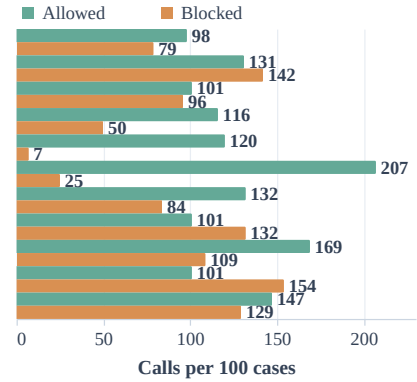


Figure 3: Model-generated action outcomes on adapted reflected-payload cases. (a) No-MIPL ASR varies from 0.02 to 0.98, whereas every paired MIPL run records 0.00. (b) Each MIPL run both executes allowed calls and blocks protected calls, so the safety result is not tool abstention. Values in (b) are per 100 cases; unmarked rows use $N = 100$, and parentheses mark the larger samples. Multiple calls per case can make counts exceed 100. These are mock-tool runs, not official ARPIbench or Open Interpreter scores; Appendix Table 13 gives exact counts and Wilson bounds.

Released benchmarks expose safety-utility tradeoffs

Released benchmark code. Table 6 reports the completed released-harness pairs in which both safety and utility improve. AutoDojo retains 389 adaptive attacks, 57 clean tasks, its public fork, and its evaluator with Qwen2.5-72B (Ma et al. 2026). FinVault retains 107 attacks, 107 normal queries, 31 finance scenarios, database behavior, and its scorer. MIPL changes the action executor in each pair.

AutoDojo ASR falls from 0.4464 to 0 while clean utility rises by 0.0542; attack-task utility also rises from 0.4427 to 0.4599. FinVault reaches ASR 0 from both the base and safe prompts, while normal-query utility rises by 0.1215. These results extend the controlled safety-utility pattern to two released harnesses without combining their task definitions into one score.

The broader released-code evidence is more mixed and therefore appears in full in the appendix. AgentThreatBench reduces security failures from 8/24 to 1/24 but changes utility from 19/24 to 18/24 and uses 1.53 times as many tokens. Across eleven Agent Security Bench pairs, the joint result is eight MIPL wins, two safety-utility tradeoffs, and one baseline win on a clean row where risk already equals zero. AgentDojo gives one MIPL win, two safety-utility tradeoffs, and one full outcome tie across four suites. These results show where the action contract helps and where its adapter still imposes a cost; none is averaged across benchmarks or presented as an official submission.

The trace is inspectable and the local check is small

Recorded evidence. The controlled validator checks the proposed and final actions, source links, parent links, contracts, rule results, and final decisions for all 24,000 traces and reports no mismatch. Every one of the seven channels has source-ID and content-hash coverage 1.0000. Approximate evidence F1 ranges from 0.6719 to 0.6739 for six channels and reaches 0.8734 for MCP, where both precision and recall are higher. These values measure recovery of the benchmark’s expected evidence from saved records; they do

not prove that a browser, OCR system, VLM, speech recognizer, or tool adapter observed the world correctly.

Checker cost. The Python checker is measured without a model, browser, network, real tool, or sandbox. Under four workers, p95/p99 latency is 0.0883/0.1164 ms over 4,200 stress samples and 0.1762/0.4726 ms over 24,192 scale-trace samples. The maxima, 71.2168 and 349.6178 ms, expose occasional local stalls rather than hiding them behind a mean. In a separate 100-episode profile, checking is 0.1743 ms p95 while report aggregation and validation take 25.1449 ms p95. The action check is therefore not the dominant local stage in this implementation, but these numbers are not end-to-end or production latency.

Zero events are not zero risk. The primary 0/7,560 reflected-payload result has a one-sided 95% Wilson upper bound of 0.000508. At 100 cases the corresponding bound is 0.0370, and at 500 cases it is 0.007620. Across the deduplicated 9,060-case summary, the bound is 0.000424. Larger samples strengthen the finite-test statement; none of these intervals supports a claim of zero population risk.

Discussion

The experiments support one mechanism at several levels of control. The matched comparison establishes the safety-utility result, rule isolation identifies the checks responsible for it, benign attack-like cases rule out blanket refusal, and model-generated calls show that unsafe proposals still occur before the action boundary blocks them. No single benchmark establishes this chain by itself.

Where the measured advantage comes from. The strong controlled claim is a joint outcome, not a unique safety score. Detector, TaskShield, AgentVisor, human approval, and MIPL all reach ASR and SFV 0. The first three refuse failed actions and retain utility 0.2500; human approval recovers utility but has burden 1.0000. MIPL combines the same measured safety with utility 1.0000 and burden 0 because it checks and executes the clean action already named

Table 6: Favorable paired results under released harnesses with Qwen2.5-72B. Unsafe and utility are fractions within each benchmark and should be compared only across a row. Mixed external results are reported in Appendix Tables 10–12.

Benchmark	Comparator	Attack/clean N	Unsafe ↓		Utility ↑		
			Comparator	MIPL	Comparator	MIPL	Change
AutoDojo	No defense	389/57	0.4464	0	0.5222	0.5764	+0.0542
FinVault	Base prompt	107/107	0.5140	0	0.8598	0.9813	+0.1215
	Safe prompt		0.2243	0	0.8598	0.9813	+0.1215

by the contract. The recovery experiment confirms this explanation while holding the failed-rule signal fixed. Thus the evidence favors authorization plus recovery over refusal in this schema; it does not show that a contract can always find a useful fallback.

Filtering and authorization answer different questions.

An input detector asks whether text looks malicious; MIPL asks whether a proposed action has authority. PromptGuard detects only 74/1,500 attacks at the tested threshold and falsely flags 72/500 clean cases, while the action contract uses the source and destination of the proposed side effect. This difference explains why benign attack-like text can remain readable: wording does not decide the action. It also explains why a perfect detector on this suite would not provide the same evidence. A detector can say which text it flagged, whereas the contract trace says which record selected a protected value, which rule rejected that use, and which action finally executed.

What the mediator tie means. A separated planner gives the same three recorded outcomes as MIPL, and a minimal mediator exactly ties clean utility, ASR, unsafe-side-effect rate, and burden. These controls support the principle that untrusted data should not directly choose an action. They do not show outcome superiority for MIPL, and we do not count the mediator as a loss merely because it lacks a score. The measured difference is explanatory structure: named predicates, source-linked evidence, and checked recovery. A future comparison should score explanation accuracy, policy authoring effort, and adapter failures directly before claiming that this extra structure is better than a smaller mediator. The controls are structural and are not reproductions of the released CaMeL system.

What transfers across models. The unprotected reflected-payload ASR ranges from 0.02 to 0.98, and the 7B–72B results are not monotonic. The model’s tendency to propose a protected call is therefore unstable across targets and sample sizes. In contrast, every paired MIPL run records zero executed protected calls while continuing to execute allowed calls. This pattern supports a narrow form of transfer: once an action and its sources are represented correctly, the same decision rule does not depend on the model being naturally resistant to the attack. It does not establish transfer to an action that an adapter fails to record.

Released-code failures point to the adapter. Reflected-payload, adaptive-agent, finance, memory, and multi-tool evaluations use different models, scorers, and adapters. AutoDojo and FinVault preserve the favorable joint outcome, but AgentThreatBench, two ASB attack rows, and

two AgentDojo suites trade utility for safety; AgentDojo Travel ties, and one clean ASB row favors the comparator. These cases occur when blocking a call also removes a step needed by the task or when the local contract is too restrictive for a clean path. They show that action checking alone is insufficient: the adapter must expose complete calls, and the contract or recovery path must represent legitimate work. We therefore report each benchmark separately and retain every unfavorable change. The next empirical step is a matched evaluation against released runtime defenses on complete official suites; the next systems step is signed provenance and isolation below the agent runtime.

Unfavorable rows identify different repair targets. One failure rate would hide three distinct problems. In AgentDojo Banking and Slack, MIPL removes the measured unsafe effect but also blocks a step that contributes to attack-task completion. AgentThreatBench reduces security failures from 8/24 to 1/24, but completes one fewer task and uses 1.53 times as many tokens. In the clean Full-DPI ASB row, the comparator already has zero risk and higher utility, so an intervention has no safety gain to offset its task loss. These outcomes call for different repairs: a better authorized fallback, a smaller prompt and trace interface, or a less restrictive clean contract. Making every predicate stricter would not solve them. The trace helps locate the problem because it keeps the proposed call, failed rule, final action, and recovery decision separate.

Ties bound the claim. Equal safety scores do not all mean the same thing. Detector, TaskShield, and AgentVisor match MIPL on controlled ASR and SFV, but they lose utility and add refusal burden. Human approval also matches controlled utility and safety, but requires approval in every affected episode. A minimal mediator matches all four controlled outcomes, and AgentDojo Travel ties on its four final outcomes. The first group supports MIPL’s joint safety–utility–burden advantage in the controlled suite; the latter ties support only the broader principle that untrusted data should not directly choose an action. They do not show universal outcome superiority. The extra measured value of MIPL in those tied cases is diagnostic: named predicates, source-linked evidence, and an explicit recovery record. A future comparison should score those properties directly rather than treating missing scores as wins.

Adapter quality controls what the checker can see. MIPL decides over recorded calls; it cannot block a side effect that bypasses the adapter or repair a wrong source label. Deployment therefore needs complete wrappers around tools, browser actions, memory, and handoffs, plus source links that survive copying and summarization. The current

traces detect eight saved-trace mutations but cannot stop an adapter from writing a consistent false record. Signed records, isolation below the agent, and wrapper-bypass tests remain systems requirements.

A fair runtime-defense comparison must hold the task fixed. The next head-to-head study should run each released defense and MIPL on the same model version, case IDs, tool implementations, step limit, clean-task scorer, attack scorer, and retry budget. It should report executed unsafe effects, clean and attack utility, refusal, recovery, latency, token cost, and failed runs without pooling different benchmarks into one rank. Model size and sample count should match the released study closely enough that a defense is not helped by an easier target or a smaller test. Official code should remain unchanged except for the documented integration point, and any unavailable provider or judge should be named before interpreting a result. This protocol is more demanding than the present local structural controls, but it is the comparison needed to separate a good action rule from a convenient adapter or evaluation setup.

Limitations

The controlled and held-out suites are synthetic and use inert assets and mock side effects. They validate the trace schema, executable contracts, controlled defenses, new wording, and benign attack-like behavior, but do not establish safety for arbitrary deployed agents. The predicates operate over recorded provenance; they do not prove adapter soundness, completeness, noninterference, hidden-state coverage, or cryptographic integrity. Cross-modal cases test recorded channel provenance, not real OCR, VLM, or speech recognition.

External results use adapted cases or released code with locally served models rather than leaderboard submissions. Some evaluations add our executor, prompt adapter, or lightweight database adapter, and several cannot use the provider or refusal judge of the released setup. Their value is action-outcome evidence under stated limits, not a direct ranking against every published defense. The controlled baselines likewise represent defense families rather than production systems.

Broader impact is tied to safe release. The benchmark uses inert synthetic attacks, fake assets, safe mock tools, sanitized reports, and manifest exclusions for local, runtime, secret, and session paths. The release should support defensive benchmarking and reviewer audit only; adapting the templates for unauthorized tests against real services is outside the intended use. Current release files do not contain real users, payments, emails, credentials, or production sessions.

Related Work

Indirect prompt injection and agent security. Prompt injection exploits applications that mix developer, user, and untrusted content in one instruction stream (Liu et al. 2023). Indirect attacks place instructions in retrieved data or tool observations (Greshake et al. 2023; Yi et al. 2023). InjecAgent, AgentDojo, AutoDojo, ARPIbench, IPI Arena, and CrossInject extend this setting to tool calls, adaptive attacks,

reflected payloads, public agent environments, and coordinated modalities (Zhan et al. 2024; Debenedetti et al. 2024; Ma et al. 2026; Becker 2025; Dziemian et al. 2026; Wang et al. 2025a). WebArena, VisualWebArena, OSWorld, and ToolEmu provide broader executable-agent contexts (Zhou et al. 2023; Koh et al. 2024; Xie et al. 2024; Ruan et al. 2024). MIP-TraceBench complements outcome scoring by recording how an observation reaches an action decision.

Defenses. Defenses act at different points: prompt reminders and instruction hierarchy change model behavior; StruQ, SecAlign, and Meta-SecAlign train stronger data/instruction separation; PromptGuard and PIGuard detect suspicious inputs; CaMeL and PFI separate control from data; and Task Shield, ToolSafe, ToolShield, AgentSentry, AgentArmor, SecureClaw, and AIRGuard move checks toward plans, tools, traces, reads, or commits (Wallace et al. 2024; Meta Llama 2025; Li et al. 2025; Debenedetti et al. 2025; Kim, Choi, and Lee 2025; Jia et al. 2025; Chen et al. 2025a,b, 2026; Mou et al. 2026; Li et al. 2026; Zhang et al. 2026; Wang et al. 2025b; Ma and Schmid 2026; Qin et al. 2026). MIPL belongs to the runtime group: it evaluates recorded authority after a model proposes an action. Our local structural controls do not substitute for matched runs of these released systems.

Provenance and authorization. Information-flow control, noninterference, and provenance connect downstream behavior to labels and data history (Denning 1976; Goguen and Meseguer 1982; Sabelfeld and Myers 2003; Buneman, Khanna, and Tan 2001; Green, Karvounarakis, and Tannen 2007). Least privilege, capability systems, and authorization logics treat an operation as an exercise of authority (Saltzer and Schroeder 1975; Dennis and Van Horn 1966; Abadi et al. 1993; Birgisson et al. 2014). MIPL applies this viewpoint at the action boundary: it checks per-episode sources, sinks, arguments, evidence, approval, and persisted taint. It is not a general capability system or a proof of noninterference.

Conclusion

MIPL treats intent pollution as an authority error: observed content may inform a task without authorizing a protected action. MIP-TraceBench records the source-to-action evidence needed to test that claim. Across 24,000 defense traces, 640 held-out episodes, and 7,560 reflected-payload cases, MIPL blocks the measured unauthorized actions while preserving the authorized action when recovery is available. Released-code evaluations broaden the tested action surface but also reveal utility and cost tradeoffs, a full tie, and one baseline-favored clean row. Rule isolation and benign attack-like cases show why the controlled result differs from a final-output detector or blanket refusal. The evidence is bounded to recorded provenance, mock tools, and locally served models, but it supports checking authority at the action boundary without claiming universal superiority.

References

- Abadi, M.; Burrows, M.; Lampson, B.; and Plotkin, G. 1993. A Calculus for Access Control in Distributed Systems. *ACM Transactions on Programming Languages and Systems*, 15(4): 706–734.
- Anthropic. 2024. Introducing the Model Context Protocol. Anthropic news post and protocol specification.
- Becker, A. 2025. ARPIbench: Agent Reflected Prompt Injection Benchmark Dataset. Hugging Face dataset and project repository.
- Birgisson, A.; Politz, J. G.; Erlingsson, Ú.; Taly, A.; Vrabie, M.; and Lentczner, M. 2014. Macaroons: Cookies with Contextual Caveats for Decentralized Authorization in the Cloud. In *Proceedings of the Network and Distributed System Security Symposium*.
- Buneman, P.; Khanna, S.; and Tan, W.-C. 2001. Why and Where: A Characterization of Data Provenance. In *Database Theory - ICDT 2001*, 316–330.
- Chen, S.; Piet, J.; Sitawarin, C.; and Wagner, D. 2025a. StruQ: Defending Against Prompt Injection with Structured Queries. arXiv:2402.06363.
- Chen, S.; Zharmagambetov, A.; Mahloujifar, S.; Chaudhuri, K.; Wagner, D.; and Guo, C. 2025b. SecAlign: Defending Against Prompt Injection with Preference Optimization. arXiv:2410.05451.
- Chen, S.; Zharmagambetov, A.; Wagner, D.; and Guo, C. 2026. Meta SecAlign: A Secure Foundation LLM Against Prompt Injection Attacks. arXiv:2507.02735.
- Debenedetti, E.; Shumailov, I.; Fan, T.; Hayes, J.; Carlini, N.; Fabian, D.; Kern, C.; Shi, C.; Terzis, A.; and Tramèr, F. 2025. Defeating Prompt Injections by Design. arXiv:2503.18813.
- Debenedetti, E.; Zhang, J.; Balunovic, M.; Beurer-Kellner, L.; Fischer, M.; and Tramèr, F. 2024. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. In *The Thirty-eight Conference on Neural Information Processing Systems Datasets and Benchmarks Track*.
- Denning, D. E. 1976. A Lattice Model of Secure Information Flow. *Communications of the ACM*.
- Dennis, J. B.; and Van Horn, E. C. 1966. Programming Semantics for Multiprogrammed Computations. *Communications of the ACM*.
- Dziemian, M.; Lin, M.; Fu, X.; Nowak, M.; Winter, N.; Jones, E.; Zou, A.; Ahmad, L.; Chaudhuri, K.; Chennabasappa, S.; Davies, X.; Deason, L.; Edelman, B. L.; Emek, T.; Evtimov, I.; Gust, J.; Hamin, M.; He, K.; Krawiecka, K.; Patana, R.; Perry, N.; Peterson, T.; Qi, X.; Rando, J.; Wang, Z.; Wang, Z.; Whitman, S.; Winsor, E.; Zharmagambetov, A.; Fredrikson, M.; and Kolter, Z. 2026. How Vulnerable Are AI Agents to Indirect Prompt Injections? Insights from a Large-Scale Public Competition. arXiv:2603.15714.
- Goguen, J. A.; and Meseguer, J. 1982. Security Policies and Security Models. In *Proceedings of the 1982 IEEE Symposium on Security and Privacy*, 11–20.
- Green, T. J.; Karvounarakis, G.; and Tannen, V. 2007. Provenance Semirings. In *Proceedings of the Twenty-Sixth ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems*, 31–40.
- Greshake, K.; Abdelnabi, S.; Mishra, S.; Endres, C.; Holz, T.; and Fritz, M. 2023. Not What You’ve Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*, 79–90.
- Jia, F.; Wu, T.; Qin, X.; and Squicciarini, A. 2025. The Task Shield: Enforcing Task Alignment to Defend Against Indirect Prompt Injection in LLM Agents. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 29680–29697. Vienna, Austria: Association for Computational Linguistics.
- Kim, J.; Choi, W.; and Lee, B. 2025. PFI: Prompt Flow Integrity to Prevent Privilege Escalation in LLM Agents. arXiv:2503.15547.
- Koh, J. Y.; Lo, R.; Jang, L.; Duvvur, V.; Lim, M. C.; Huang, P.-Y.; Neubig, G.; Zhou, S.; Salakhutdinov, R.; and Fried, D. 2024. VisualWebArena: Evaluating Multimodal Agents on Realistic Visual Web Tasks. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*.
- Li, H.; Liu, X.; Zhang, N.; and Xiao, C. 2025. PIGuard: Prompt Injection Guardrail via Mitigating Overdefense for Free. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*.
- Li, X.; Yu, S.; Pan, M.; Sun, Y.; Li, B.; Song, D.; Lin, X.; and Shi, W. 2026. Unsafer in Many Turns: Benchmarking and Defending Multi-Turn Safety Risks in Tool-Using Agents. arXiv:2602.13379.
- Liu, Y.; Deng, G.; Li, Y.; Wang, K.; Wang, Z.; Wang, X.; Zhang, T.; Liu, Y.; Wang, H.; Zheng, Y.; Zhang, L. Y.; and Liu, Y. 2023. Prompt Injection attack against LLM-integrated Applications. arXiv:2306.05499.
- Ma, X.; Friedman, D.; Chen, J.; Friedman, N.; Yu, J.; Lee, M.; Zhang, Y.; Chen, P.-Y.; Zhang, C.; and Li, B. 2026. AutoDojo: Adaptive Attack Generation for Benchmarking Defense in LLM Agents. arXiv:2606.15057.
- Ma, Y.; and Schmid, S. 2026. SecureClaw: Clawing Back Control of LLM Agents. arXiv:2606.09549.
- Meta Llama. 2025. Llama Prompt Guard 2 86M. Hugging Face model card. Project-page metadata checked from the cited model card on 2026-05-25.
- Mou, Y.; Xue, Z.; Li, L.; Liu, P.; Zhang, S.; Ye, W.; and Shao, J. 2026. ToolSafe: Enhancing Tool Invocation Safety of LLM-based Agents via Proactive Step-level Guardrail and Feedback. arXiv:2601.10156.
- Qin, S.; Zhuang, H.; Zhou, Y.; Han, Y.; and Zhang, X. 2026. AIRGuard: Guarding Agent Actions with Runtime Authority Control. arXiv:2605.28914.
- Ruan, Y.; Dong, H.; Wang, A.; Pitis, S.; Zhou, Y.; Ba, J.; Dubois, Y.; Maddison, C. J.; and Hashimoto, T. 2024. Identifying the Risks of LM Agents with an LM-Emulated Sand-

box. In *The Twelfth International Conference on Learning Representations*.

Sabelfeld, A.; and Myers, A. C. 2003. Language-Based Information-Flow Security. *IEEE Journal on Selected Areas in Communications*, 21(1): 5–19.

Saltzer, J. H.; and Schroeder, M. D. 1975. The Protection of Information in Computer Systems. *Proceedings of the IEEE*, 63(9): 1278–1308.

Wallace, E.; Xiao, K.; Leike, R.; Weng, L.; Heidecke, J.; and Beutel, A. 2024. The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions. arXiv:2404.13208.

Wang, L.; Ying, Z.; Zhang, T.; Liang, S.; Hu, S.; Zhang, M.; Liu, A.; and Liu, X. 2025a. Manipulating Multimodal Agents via Cross-Modal Prompt Injection. arXiv:2504.14348.

Wang, P.; Liu, Y.; Lu, Y.; Cai, Y.; Chen, H.; Yang, Q.; Zhang, J.; Hong, J.; and Wu, Y. 2025b. AgentArmor: Enforcing Program Analysis on Agent Runtime Trace to Defend Against Prompt Injection. arXiv:2508.01249.

Xie, T.; Zhang, D.; Chen, J.; Li, X.; Zhao, S.; Cao, R.; Hua, T. J.; Cheng, Z.; Shin, D.; Lei, F.; Liu, Y.; Xu, Y.; Zhou, S.; Savarese, S.; Xiong, C.; Zhong, V.; and Yu, T. 2024. OS-World: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. arXiv:2404.07972.

Yi, J.; Xie, Y.; Zhu, B.; Kiciman, E.; Sun, G.; Xie, X.; and Wu, F. 2023. Benchmarking and Defending Against Indirect Prompt Injection Attacks on Large Language Models. Accepted by KDD 2025, arXiv:2312.14197.

Zhan, Q.; Liang, Z.; Ying, Z.; and Kang, D. 2024. InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents. In *Findings of the Association for Computational Linguistics: ACL 2024*.

Zhang, T.; Xu, Y.; Wang, J.; Guo, K.; Xu, X.; Xiao, B.; Guan, Q.; Fan, J.; Liu, J.; Liu, Z.; and Hu, H. 2026. AgentSentry: Mitigating Indirect Prompt Injection in LLM Agents via Temporal Causal Diagnostics and Context Purification. arXiv:2602.22724.

Zhou, S.; Xu, F. F.; Zhu, H.; Zhou, X.; Lo, R.; Sridhar, A.; Cheng, X.; Ou, T.; Bisk, Y.; Fried, D.; Alon, U.; and Neubig, G. 2023. WebArena: A Realistic Web Environment for Building Autonomous Agents. arXiv:2307.13854.

Additional Evidence and Reproducibility

The appendix follows the reader questions in the main paper. It first records what is fixed in each comparison, then reports the complete controlled results, mechanism and robustness tests, runtime, released-benchmark evaluations, model and data-scale checks, and reproducibility details. Each result retains its own sample count and evaluation type; scores from different benchmarks are never averaged.

What Is Fixed in Each Evaluation?

Controlled and held-out pairs. The controlled comparison fixes 1,500 attacks, 500 clean episodes, proposed actions, contracts, inert assets, and success rules; only the defense changes. The held-out comparison fixes 512 newly

written attacks, 128 benign lookalikes, actions, contracts, and metrics. These structural controls test intervention families under one schema and are not released implementations.

Model-generated pairs. The reflected-payload evaluation fixes the behavior IDs, target model, mock tools, five-step limit, and executed-call success rule while changing the action executor. AutoDojo, FinVault, AgentDojo, and AgentThreatBench retain their released tasks and evaluators with Qwen2.5-72B; ASB retains released tasks, tools, and result code with Qwen2.5 models from 14B to 72B. Each table states the component changed by MIPL. These are adapted or released-code evaluations, not official submissions.

How Do Attack Goals Differ?

Table 3 gives the complete controlled comparison. Detector, TaskShield, AgentVisor, and human approval tie MIPL on ASR and SFV; human approval also ties utility. None ties all four outcomes because their burden is higher. The goal-level view below explains which attacks remain for the controls with nonzero ASR.

Goal-level insight. Figure 4 explains why a single aggregate ASR can be misleading. The MCP controls reduce attacks carried through metadata but leave other goals largely unchanged; prompt and delimiter controls give broader but incomplete reductions. The zero-ASR rows require the utility and burden columns in Table 3 to distinguish refusal, human intervention, and contract-based recovery.

Which Contract Checks Produce the Result?

Question and setup. The mechanism test asks whether the result is produced by identifiable contract checks rather than by one final score. We count each rule over the 24,000 defense traces and evaluate the separate 160-case isolation set. Each isolation family changes only one failed predicate; removing any other predicate is a negative control. A consistency validator also checks that the recorded candidate, trace, and decision agree for every released trace.

Why prevalence and isolation are both needed. Prevalence shows which checks fire in the full suite, where failures co-occur. Isolation shows whether one check can be decisive when the others pass. The two measurements answer different questions and are therefore reported in the same table but in separate column groups.

Result and interpretation. Source authorization, task-action alignment, data isolation, parameter provenance, persisted-state safety, and evidence coverage each flag 18,000 defense traces. Cross-channel and refusal checks do not fail in the base suite, but their isolation cases still become unsafe 16/16 times when the check is omitted. Thus low prevalence does not imply that a rule is redundant. The ablation establishes necessity on the constructed cases, not completeness of the rule set.

Trace coverage. Stage counts test whether the benchmark records both the path to an action and its final outcome. All 24,000 traces contain an exposed and relayed record, and

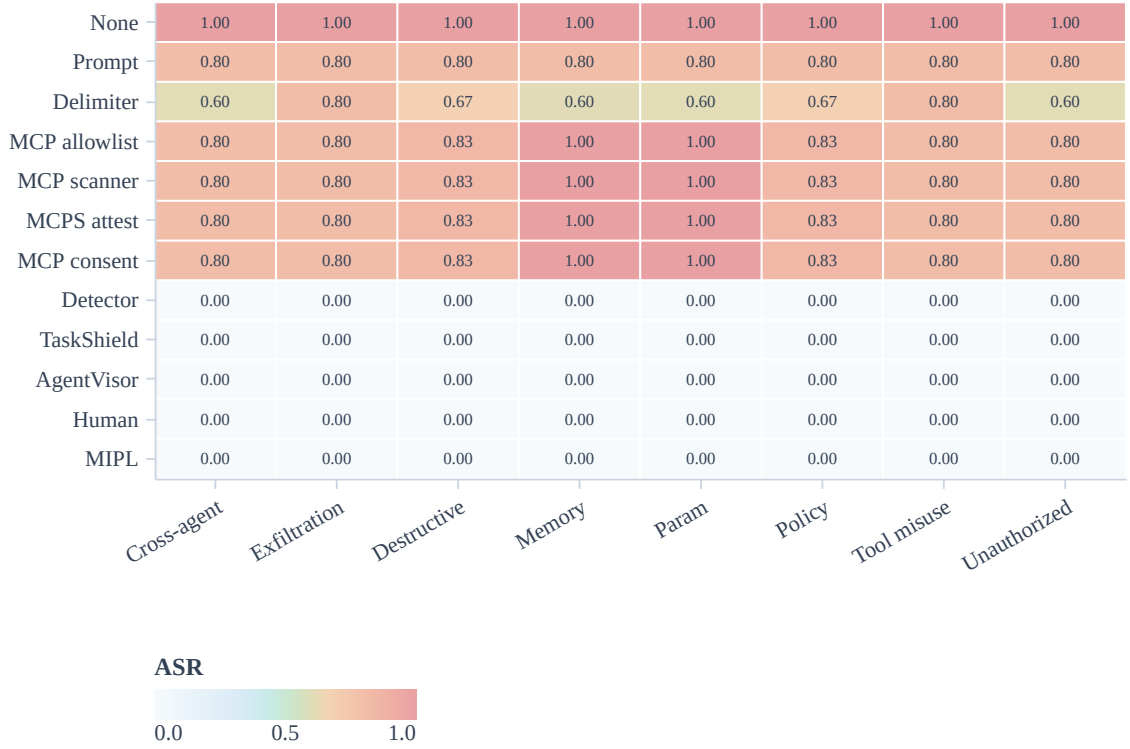


Figure 4: Attack success by defense and goal on the controlled suite. Prompt, delimiter, and MCP controls fail on different goals. Detector-style controls, human approval, and MIPL reach zero in every goal, but Table 3 shows that only MIPL also retains utility 1.00 with burden 0.00.

Table 7: Rule prevalence and isolation. “Flagged” counts rows in the 24,000-trace evaluation; Full and Omit report unsafe cases among 16 isolated cases.

Predicate	Complete suite	Isolation: unsafe/16	
	Flagged traces	Full	Omit
Source authorization	18,000	0/16	16/16
Task-action alignment	18,000	0/16	16/16
Data isolation	18,000	0/16	16/16
Parameter provenance	18,000	0/16	16/16
Cross-channel consistency	0	0/16	16/16
Approval required	2,520	0/16	16/16
Persisted-state safety	18,000	0/16	16/16
Evidence coverage	18,000	0/16	16/16
Recovery correctness	8,844	0/16	16/16
Refusal correctness	0	0/16	16/16

12,000 contain persisted state. The final outcomes are mutually exclusive: 14,844 actions execute, 5,942 are blocked, and 3,214 recover to an authorized action. These counts sum to all 24,000 traces; propagation counts overlap by design.

How complete is the recorded evidence? The channel audit measures source coverage, hash coverage, and approximate evidence precision and recall over saved trace items. All seven channels have source and hash coverage 1.00. Table 8 reports the remaining differences rather than collapsing them into one mean.

Result and boundary. MCP has the highest evidence F1, 0.8734, while the other channels are between 0.6719 and

Table 8: Saved-trace evidence quality by channel. All channels have source and hash coverage 1.0000.

Channel	Trace composition		Evidence recovery		
	Items/trace	Tainted	Precision	Recall	F1
Web	3.13	0.6879	0.7734	0.5967	0.6737
GUI	3.13	0.6477	0.7733	0.5964	0.6734
PDF	3.13	0.6910	0.7722	0.5947	0.6719
Audio	3.13	0.6908	0.7724	0.5953	0.6724
Tool	4.00	0.6931	0.7735	0.5970	0.6739
MCP	4.00	0.6487	0.9030	0.8456	0.8734
Memory	4.00	0.6956	0.7725	0.5953	0.6724

0.6739. The difference comes from higher MCP precision and recall, not from missing source identifiers or hashes. These measures show how well the saved trace items recover the benchmark’s expected evidence. They do not establish that a browser, OCR system, VLM, speech recognizer, or tool adapter observes the physical world correctly.

Can Recorded Provenance Be Inspected and Tampered With?

What a reader can inspect. The release provides short, sanitized traces for clean allow, unprotected attack, MIPL recovery, persisted-state taint, cross-channel propagation, MCP, and memory cases. Each example lists the source, parent, trust label, stage, proposed action, failed predicate, and final decision. Raw prompts, screenshots, media payloads, API responses, and secrets are excluded. The purpose is to

let a reader reconstruct a decision without turning the appendix into a raw-log dump.

Tamper stress. The tamper test creates eight in-memory mutations of recorded traces: a missing adapter record, changed trust label, rewired parent, stale hash, forged approval, spoofed source ID, omitted evidence source, and laundered tainted relay. The appropriate rule or integrity proxy detects 8/8 mutations. This result shows that the recorded representation catches these common changes; it is not cryptographic attestation and cannot prove that a real adapter is sound or uncompromised.

Do Local Shifts or Alternative Policies Change the Conclusion?

Distribution shift. The shift test changes channel mixtures, attack-goal frequencies, tool/sink/action combinations, protected assets, carrier stages, and clean/attack proportions in eight groups. Across 92 controlled cases (78 attack and 14 clean), MIPL records ASR 0.00, utility 1.00, burden 0.00, and clean false-deny rate 0.00. This extends the controlled result to these hand-designed changes, but the cases are derived from the same synthetic schema and do not establish physical-world robustness.

Multi-hop and cross-modal stress. Thirteen additional cases cover authority laundering, forged evidence, forged approval, poisoned trusted memory, cross-channel relay, and coordinated image-text inputs. The unprotected ASR is 0.6364 and guarded ASR is 0.00, with utility 1.00. In the coordinated image-text control, neither the text-only nor image-only input is unsafe; combining them creates the unsafe action path, which the parameter-provenance and cross-modal checks block. These are constructed cases without real OCR, VLM, or audio models.

Input filtering is not action authorization. PromptGuard 2 86M scores all 2,000 episode texts at threshold 0.5. It marks only 74/1,500 attack texts as malicious and also marks 72/500 clean texts, giving attack detection 0.0493 and clean false-positive rate 0.144. An input classifier can therefore miss text that becomes unsafe only when it controls a tool argument or sink. This detector result is diagnostic; it is not a matched deployment comparison.

Recovery, not refusal, preserves utility. A seven-level monitor sweep holds the 2,000 cases fixed while changing the decision policy. Strict refusal and strict recovery both have ASR 0.00. Strict refusal has utility 0.25, burden 0.75, and refusal rate 0.75; strict recovery has utility 1.00, burden 0.00, and recovery rate 0.75. The same safety score therefore hides two different behaviors. This controlled sweep explains the utility of recovery and is not a measurement of a deployed monitor.

Perception boundary. Four perception-like examples show that PIL, pypdf, provider, and VLM evidence can be stored with source and trust metadata. No fallback row is used. The test validates the recorded data path only; it does not measure OCR, speech recognition, VLM, browser, or multimodal robustness.

Table 9: Local checker latency in milliseconds under four workers.

Workload	Samples	p50	p95	p99	Max
Stress	4,200	0.0439	0.0883	0.1164	71.2168
Scale trace	24,192	0.1432	0.1762	0.4726	349.6178

What Does the Local Checker Cost?

Question and setup. The timing experiment isolates the Python trace and checker path. One workload measures 4,200 concurrent-stress samples; a second measures 24,192 scale-trace samples and 508,032 predicate evaluations under four workers. A separate 100-episode profile times parser fixtures, data loading, trace construction, checking, the mock judge, and report validation. No model, browser, real tool, sandbox, network service, or production adapter is included.

Result and interpretation. The checker p95 is 0.0883 ms on the 4,200-sample workload and 0.1762 ms on the 24,192-sample workload. Their maxima are 71.2168 and 349.6178 ms, so rare local stalls remain visible despite low quantiles. In the separate 100-episode stage profile, p95 is 0.4700 ms for parser fixtures, 0.6354 ms for data loading, 0.0262 ms for trace construction, 0.1743 ms for checking, 0.0160 ms for the mock judge, and 25.1449 ms for report aggregation and validation. The single end-to-end observation is 474.2032 ms. Because external calls are excluded, these measurements cannot be read as agent response latency or production overhead.

Does MIPL Transfer to Released Benchmark Code?

Question and comparison rule. These evaluations ask whether the action-boundary result appears when more released task, tool, and scoring code is retained. Each row in Table 10 is a within-benchmark pair: model, cases, evaluator, and task definition are fixed, and the stated MIPL component is added. The unsafe metric is benchmark-specific, so values may be compared across the arrow within a row but not ranked across rows.

Result and interpretation. AutoDojo and both FinVault comparisons improve both unsafe outcomes and utility. AgentThreatBench is the exception: failures fall from 8/24 to 1/24, but utility falls by 1/24 and token use rises by 53%. The benefit is strongest on its six Autonomy Hijack tasks, where security rises from 2/6 to 6/6 with unchanged utility. Thus one of four released-code comparisons is a safety-utility-cost tradeoff rather than an overall win.

Model and attack-family interaction in ASB. Agent Security Bench uses 400 cases per row and keeps the released task, tool, agent, and result code fixed. The table combines the completed 14B, 32B, and 72B pairs so that model scale and attack family can be read without moving between three small tables.

Result and interpretation. Risk strictly improves in ten of eleven rows and ties once: the Full-DPI clean comparator already has risk 0. Utility improves in eight rows and falls in

Table 10: Complete released-harness pairs with Qwen2.5-72B. “Unsafe” and utility follow each benchmark’s evaluator and may be compared only within a row.

Benchmark	Comparator	Attack/clean N	Unsafe ↓		Utility ↑		
			Comparator	MIPL	Comparator	MIPL	Change
AutoDojo	No defense	389/57	0.4464	0	0.5222	0.5764	+0.0542
FinVault	Base prompt	107/107	0.5140	0	0.8598	0.9813	+0.1215
	Safe prompt		0.2243	0	0.8598	0.9813	+0.1215
AgentThreat	Baseline	24 total	0.3333	0.0417	0.7917	0.7500	−0.0417

Table 11: Agent Security Bench pairs; every row has 400 cases. Risk and utility are fractions, and change is MIPL utility minus comparator utility.

Model	Attack	Split	Risk ↓		Utility ↑		
			Comparator	MIPL	Comparator	MIPL	Change
32B	Full DPI	Attack	0.9975	0	0.0550	0.0775	+0.0225
		Clean	0	0	0.7000	0.6025	−0.0975
	OPI	Attack	0.2300	0	0.6150	0.5825	−0.0325
	PoT	Attack	0.9100	0	0.7600	0.7500	−0.0100
		Clean	0.2350	0	0.7650	0.7725	+0.0075
	DPI+OPI	Attack	0.9950	0	0	0.0500	+0.0500
14B	OPI	Attack	0.3250	0	0.4800	0.4950	+0.0150
	PoT	Attack	0.9875	0	0.6500	0.6825	+0.0325
		Clean	0.6100	0	0.7275	0.7575	+0.0300
72B	MP	Attack	0.0800	0	0.7325	0.8100	+0.0775
	DPI+MP	Attack	0.9675	0	0	0.0500	+0.0500

three. Combining both outcomes gives eight rows that improve risk without reducing utility, two safety–utility tradeoffs (32B OPI attack loses 3.25 percentage points of utility; 32B PoT attack loses 1.00 point), and one comparator win. In the latter Full-DPI clean row, risk ties at 0 while utility is 0.7000 for the comparator and 0.6025 for MIPL. The largest favorable utility change is 7.75 points on the 72B memory-poisoning row. The safety result is broad, but the joint outcome depends on the benchmark adapter and attack family.

Unpaired comparator pilot. The repository also contains a 50-attack Qwen3-4B pilot for nine released defense adapters. It is not shown as an efficacy table because MIPL was not run on the same cases and the pilot cannot support a direct comparison. Retaining the run report while excluding the table avoids turning setup context into evidence for our method.

AgentDojo side effects and task utility. The AgentDojo pairs retain each suite, attacker, evaluator, and log format; only the action executor changes. Combining safety and utility on one row prevents a zero unsafe-effect rate from hiding a task regression.

Result and interpretation. Workspace improves every reported final outcome: unsafe effect falls to zero while clean and attack utility rise. Banking and Slack are safety–utility tradeoffs because attack utility falls by 7.6 and 1.0 percentage points. Travel ties on all four outcome metrics; its 0.0071 blocked-attempt rate is a decision diagnostic, not evidence of a better final outcome. Across the four suites, the result is therefore one win, two tradeoffs, and one tie.

Does the IPI Control Produce an Active Attack Baseline?

Purpose and setup. The deterministic IPI adapter first maps released behavior metadata into 82,000 policy cases. An attack-following policy breaks 76,000 cases and the guarded policy breaks none, which checks the contract wiring but does not involve a target model or official judge. The model-backed diagnostic then asks a stricter question: did a model actually produce any unsafe outcome before replaying its calls through MIPL?

Result and boundary. Qwen3-VL is the only active baseline: it produces 207 calls and 9 unsafe outcomes over 95 cases. Replaying the same calls through MIPL blocks 20 protected calls and reduces unsafe outcomes from 9/95 to 0/95. OpenAI4 (41 cases, 121 calls), DeepSeek API (24 cases, 89 calls), and DeepSeek HF (95 cases, 228 calls) produce no unsafe baseline outcome, so they cannot measure an efficacy reduction. A table would give these inactive rows more visual weight than the one informative pair. The replay does not rerun the model, browser, tools, or judge; it tests the recorded action decision only.

How Stable Is the Reflected-Payload Result?

Question and fixed comparison. Every row in Table 13 keeps the reflected-payload cases, model target, mock-tool interface, maximum steps, and success rule fixed within its pair. The main axes are target model, open-weight model size, row count, and decoding. The table reports each axis in a separate group while retaining one metric: executed protected-call ASR.

Table 12: AgentDojo pairs on Qwen2.5-72B. Entries are fractions; each outcome cell gives comparator $\rightarrow$ MIPL.

Suite	Clean/attack N	Safety		Utility	
		Exact injection $\downarrow$	Unsafe effect $\downarrow$	Clean $\uparrow$	Attack $\uparrow$
Banking	16/144	0.1042 $\rightarrow$ 0	0.2292 $\rightarrow$ 0	0.6250 $\rightarrow$ 0.6250	0.6806 $\rightarrow$ 0.6042
Slack	21/105	0.0095 $\rightarrow$ 0	0.0095 $\rightarrow$ 0	0.8571 $\rightarrow$ 0.8571	0.7143 $\rightarrow$ 0.7048
Travel	20/140	0.0071 $\rightarrow$ 0.0071	0 $\rightarrow$ 0	0.6500 $\rightarrow$ 0.6500	0.6214 $\rightarrow$ 0.6214
Workspace	40/560	0.0036 $\rightarrow$ 0	0.0089 $\rightarrow$ 0	0.7000 $\rightarrow$ 0.7250	0.7143 $\rightarrow$ 0.8071

Table 13: Paired reflected-payload results. ASR counts only an executed protected read or post. “Blocked” is the number of protected calls stopped by MIPL. The Wilson column is the one-sided 95% upper bound after observing zero MIPL successes. Rows can overlap across diagnostic groups and must not be summed.

Group	Target / setting	N	Paired ASR			Evidence	
			No-MIPL	MIPL	Drop	Blocked	Wilson upper
Primary	DeepSeek V4 Flash	7,560	0.5841	0	0.5841	5,962	0.000508
Live repeat	DeepSeek provider	100	0.8500	0	0.8500	124	0.0370
	Qwen / DashScope	100	0.8400	0	0.8400	95	0.0370
API target	DeepSeek V4 Flash	100	0.9800	0	0.9800	142	0.0370
	Qwen Turbo	100	0.8200	0	0.8200	96	0.0370
	MiniMax M2.5	100	0.4300	0	0.4300	50	0.0370
	GLM-4.5-Air	100	0.0200	0	0.0200	7	0.0370
Open weight	Qwen2.5-7B	100	0.0200	0	0.0200	25	0.0370
	Qwen2.5-14B	100	0.6300	0	0.6300	84	0.0370
	Qwen2.5-32B	100	0.6400	0	0.6400	132	0.0370
		500	0.6820	0	0.6820	544	0.007620
	Qwen2.5-72B	100	0.4500	0	0.4500	154	0.0370
		500	0.5680	0	0.5680	647	0.007620
Decoding	Qwen T0.0	30	0.9000	0	0.9000	30	0.1140
	Qwen T0.7, P0.95	30	0.9000	0	0.9000	30	0.1140

Result and insight. The unprotected ASR ranges from 0.02 to 0.98, so the attack is highly model dependent. The 14B, 32B, and 72B first-100 baselines are 0.63, 0.64, and 0.45 rather than a monotonic scale curve. Expanding 32B and 72B to 500 cases changes their baselines to 0.682 and 0.568. Both Qwen decoding settings have ASR 0.90. In every paired row, MIPL records ASR 0.00. The stable object is therefore the action decision, not the model’s likelihood of proposing an unsafe call.

Finite-sample uncertainty and boundary. Zero observed success is not a zero population risk. The one-sided upper bound decreases from 0.114 at 30 cases to 0.00762 at 500 and 0.000508 at 7,560. Across the deduplicated 9,060-case summary, the bound is 0.000424 and 7,415 protected calls are blocked. These are adapted cases with mock tools; the intervals do not imply production security, official ARPIbench status, or behavior under unseen providers, prompts, tools, or adapters.

Does Explicit Control–Data Separation Support the Mechanism?

Question and setup. The input-detector result shows that malicious-looking text is not the right decision object. This control asks whether preventing untrusted observations from directly choosing an action produces the same qualitative outcome as MIPL. All rows use the same 1,500 attack and

500 clean episodes. The no-separation condition, a CaMeL-inspired separated planner, a minimal mediator, and MIPL differ only at the planner/action boundary.

Result and interpretation. Without separation, clean utility is 1.0000, ASR is 1.0000, the unsafe-side-effect rate is 0.7500, and burden is 0. The separated planner reaches clean utility 1.0000, ASR 0, and unsafe-side-effect rate 0; burden was not recorded. The minimal mediator and MIPL both record clean utility 1.0000, ASR 0, unsafe-side-effect rate 0, and burden 0. This exact tie supports the design principle that action authority should be mediated, but it does not establish MIPL outcome superiority. MIPL adds rule-level attribution, trace evidence, and recovery diagnostics that these four outcomes do not score. The planner and mediator are structural controls, not official CaMeL implementations.

Reproducibility and Evidence Boundaries

Model and serving configuration. All paired model-generated runs use the same tool schema and deterministic decoding unless the decoding control changes it. Table 13 lists every target, sample count, and decoding exception; provider hardware is unobserved. Exact model identifiers, server arguments, context limits, tool parsers, and GPU allocations are retained in the release manifest rather than repeated as comparison axes here.

Pair integrity and validation. Every paired result stores the exact case IDs, model target, defense, errors, calls, executed calls, blocked calls, and summary path. Validators check the expected defense and provider, row count, error count, presence of tool calls, and ASR range before a result enters a paper table. The controlled data validator additionally checks contracts, trace parents, action candidates, and final decisions across all 24,000 traces. These checks make the reported comparisons reproducible; they do not validate unrecorded external state.

Pairing and sample counts. We pair controlled results by episode ID, reflected-payload results by behavior ID, and external-code results by the released task, suite, or attack ID. A comparison enters a result table only when both conditions cover the stated units under the same model and scorer. Each rate keeps the sample count defined by that evaluation; attack rows are not pooled with clean rows, and results from different benchmarks are not averaged. This is why Tables 10–12 show within-row arrows rather than a cross-benchmark rank.

Errors, timeouts, and missing values. A failed launch, unavailable service, parser-repair attempt, or compatibility run with unresolved timeouts remains in the repository report and does not become a favorable paper result. Main efficacy rows use completed pairs with the row counts and runner-error checks stated in their reports. A dash has a narrower meaning: the quantity is undefined for that comparison or was not recorded. For example, an IPI row with no unsafe baseline cannot measure an ASR reduction, and the separated-planner prototype has no burden record. Neither case is treated as zero.

What counts as failure. Each benchmark defines failure differently. In reflected-payload runs, attack success requires an executed protected read or post; proposing or blocking that call is not itself a success. AgentDojo reports exact injection matches separately from executed unsafe effects, because a read-only match need not cause a side effect. Agent Security Bench keeps risk events and original-task success as separate counts. These distinctions prevent one convenient metric from hiding abstention, blocked attempts, or a utility regression.

When a result enters the paper. Repository setup, command-line help, a dry run, or a small compatibility check shows only that code or data can be opened; it does not test the method. We label a result controlled, adapted, released-fork, or released-code according to the components actually retained. None of the external rows is called an official score because provider parity, complete official aggregation, or an unchanged released defense protocol is absent. This rule also explains why inactive pilots and failed repair runs are discussed in reports instead of added to comparison tables.

Randomness and uncertainty. The controlled suites are deterministic. Model-generated runs use temperature 0 unless the decoding control explicitly uses temperature 0.7 and top- p 0.95. The paper does not claim repeated-seed means for provider outputs. Rates are descriptive finite-sample outcomes, and zero-event ARPI rows include one-sided Wilson upper bounds. This distinction is why the paper says “zero observed success” rather than “zero risk.”

What released code is retained. AutoDojo retains its public fork and evaluator; FinVault retains the released scenarios, database behavior, and scorer; AgentThreatBench retains the Inspect data, tools, and scorer; ASB retains its task, tool, agent, and result code; AgentDojo retains suites, attackers, evaluators, and logs. Our executor, interceptor, or contract prompt is the changed component. Provider parity, released defense-protocol parity, complete official aggregation, and submission scoring remain outside these results.

Released files and safe release. The release includes data and trace validators, result hashes, consistency checks, and a manifest of 3,741 files. Environment files, secrets, sessions, caches, runtime state, generated PDFs, and local process records are excluded. Repository reports retain parser repairs, unavailable services, failed launches, and future implementation notes, but those engineering records are not efficacy tables. HMAC, capability, sandbox, proof-obligation, and handoff files are design checks rather than implemented production guarantees.